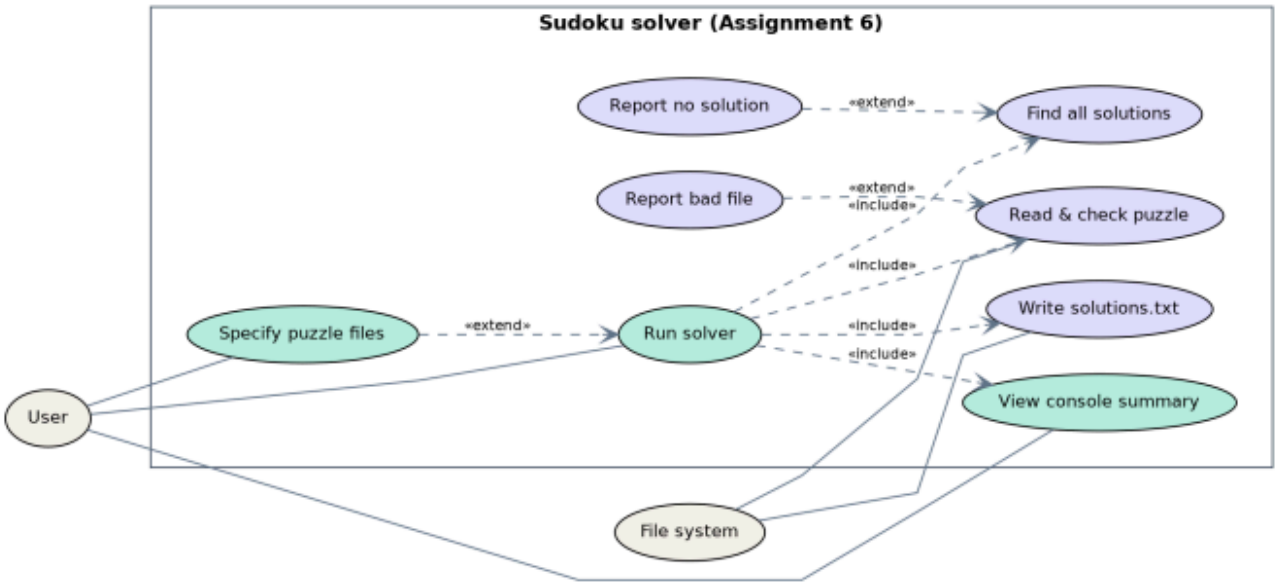


a. Identify/Discover the Use Cases

The main use case of the program is to run a Sudoku solver that reads puzzle files, checks whether they are valid, and finds all possible solutions. The other use cases include specifying puzzle files, reading and checking the puzzles, finding all solutions, writing the results to `solutions.txt`, reporting invalid puzzle files, reporting puzzles with no solutions, and displaying a console summary.

The two actors involved are the User, who runs the program and can specify puzzle files, and the File System, which provides the input files and stores the output files.

b. Brief Description of the Use Case

The purpose of the Sudoku solver is to read five Sudoku puzzle files and determine all possible solutions for each puzzle. The program uses recursive backtracking and bitmasks to check whether numbers can be placed in certain positions without violating Sudoku rules. Once the puzzles have been processed, the program writes all the solutions into a single file named `solutions.txt`, including the puzzle name and the number of solutions found. If a puzzle has no solution or contains conflicting numbers, the program reports that no solution was found.

c. Step-by-Step Flow of Events

1. The user runs the Sudoku solver program.

2. The program checks whether the user provided any puzzle filenames. If not, it automatically uses `puzzle1.txt` through `puzzle5.txt`.
3. The program opens `solutions.txt` to store the results from all the puzzles.
4. The program reads each puzzle file and checks whether it contains 81 valid entries.
5. The program checks for conflicting numbers in the rows, columns, and 3×3 boxes.
6. If the puzzle is valid, the program uses recursive backtracking to find all possible solutions.
7. Each solution is written to `solutions.txt` and the corresponding individual `solutionN.txt` file.
8. The program records the total number of solutions found for each puzzle.
9. If no solutions are found, the program writes `No solution found.` instead.
10. The program displays a summary in the console and continues until all five puzzles have been processed.

d. Detailed Description of the Use Case

Use Case Name: Run Sudoku Solver

Primary Actor: User

Supporting Actor: File System

Preconditions: The program must be compiled and executable, and the puzzle files should be available in the correct format. Each puzzle should contain 81 entries consisting of numbers 1 through 9 or underscores representing empty cells.

Main Flow: The user starts the program, which automatically accesses the five default puzzle files unless other filenames are provided. Each puzzle is read and checked for validity before the program begins solving it. The solver uses recursive backtracking to test possible values for empty cells while using bitmasks to keep track of numbers already used in each row, column, and box. When a valid solution is found, it is written to the output files. After all possible solutions have been found, the program records the total number of solutions and moves on to the next puzzle. Once all files have been processed, the program finishes.

Alternative Flows: If a puzzle contains conflicting numbers or has no possible solution, the program reports `No solution found.` If a file is missing or incorrectly formatted, the program records the puzzle as having no solution in `solutions.txt` and continues processing the remaining files. If an individual solution file cannot be created, the program still attempts to write the puzzle results to `solutions.txt`. However, if `solutions.txt` cannot be created, the program displays an error and stops.

Postconditions: The program produces a `solutions.txt` file containing the results for each processed puzzle, including its filename, all possible solutions, and the number of solutions found. It also produces individual solution files when those files can be created.